

Formation FPGA

TP3 – Machine à états (FSM)

1) En reprenant le fichier VHDL du TP2, je crée le fichier *counter_unit.vhd*.

Le code du fichier définit une entité simplifiée par rapport à celle du TP2.

Elle possède toujours 2 entrées (*clk* et *resetsn*), mais plus qu'une seule sortie, *end_counter*. La sortie *led* qui existait dans le TP précédent est supprimée, ainsi que toute la logique l'accompagnant (détection du front montant sur le signal interne *r_cible*, stockage et mise-à-jour du signal *led_in*).

La variable *cible* (ie le nombre de coups d'horloge entre deux pics de *end_counter*) était une constante lors du TP2. Il s'agit maintenant d'un paramètre *generic()* (de type positive, maximum 31 bits), que l'on peut redéfinir pour chaque instanciation de *Counter_unit*.

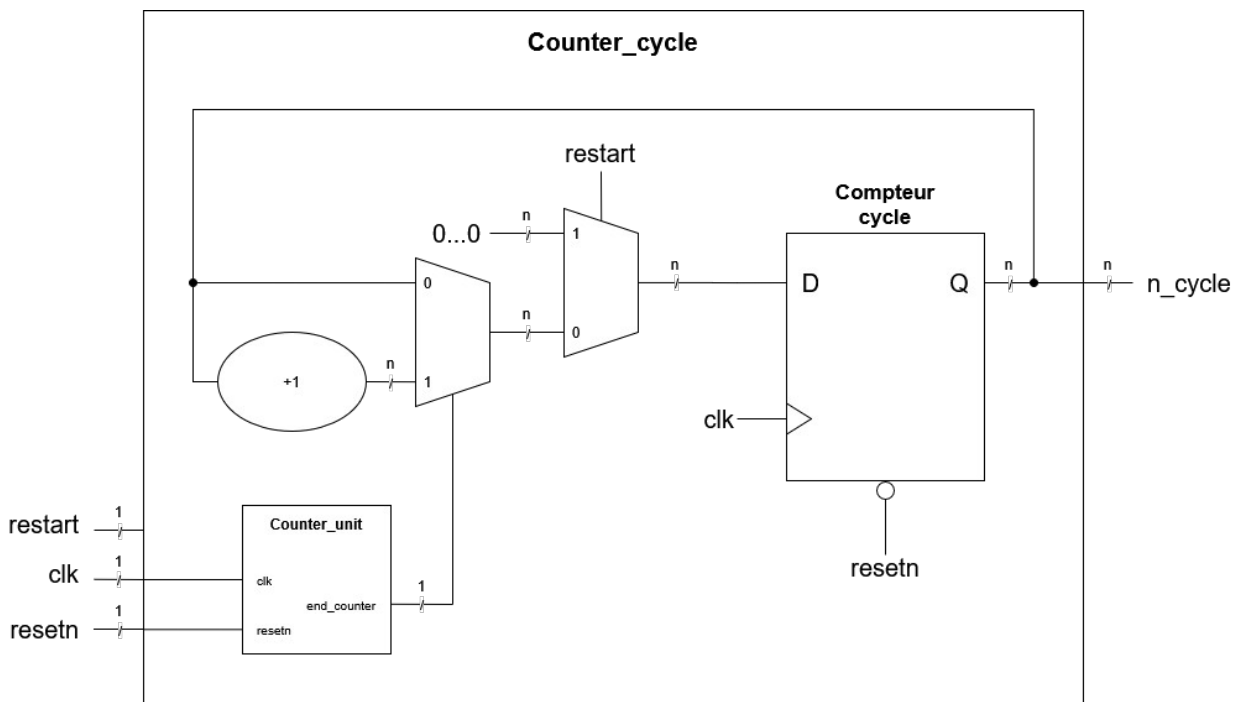
Pour adapter la taille du signal *Data*, on rajoute une ligne pour calculer la taille du vecteur nécessaire pour stocker l'information du compteur (les fonctions *ceil* et *log2* viennent de la bibliothèque *ieee.math_real* :

```
constant n_bits    : integer := integer(ceil(log2(real(cible))));
signal Data        : unsigned (n_bits-1 downto 0) := (others => '0');
```

On adapte aussi la réinitialisation du signal *Data*, pour qu'elle ne soit plus assurée sur un signal fixe, mais en réinitialisant à '1' le dernier bit et à '0' les autres :

```
if (r_cible = '1') then
    Data(n_bits-1 downto 1) <= (others => '0');
    Data(0) <= '1';
```

2) Voici le schéma RTL de notre compteur du signal *end_counter* :



Ce schéma comporte trois entrées :

- *clk*, le signal d'horloge (sur 1 bit) ;
- *resetn*, le signal de reset haut global (sur 1 bit) ;
- *restart*, un signal qui permet de réinitialiser le registre de notre compteur de cycles (sur 1 bit).

Il comporte une sortie : *n_cycle* (sur un nombre variable *n* de bits, paramètre *generic()* défini à chaque instantiation) qui correspond au nombre de pics en sortie d'une instance *Counter_unit* depuis le dernier restart/resetn.

L'architecture décrite par le schéma comprend :

- Une instance de *Counter_unit*, pilotée par l'horloge *clk* et le signal *resetn*.
- Un registre enregistrant les *n* bits du signal *n_cycle*, piloté par l'horloge *clk* et le signal *resetn*.
- La logique de mise à jour en entrée registre, qui garde sa valeur actuelle lorsque la sortie *end_counter* de l'instance *Counter_unit* vaut '0', et s'incrémente de 1 lorsque la sortie *end_counter* de l'instance *Counter_unit* vaut '1'.
- La logique de réinitialisation synchrone du compteur, qui se remet à 0 lorsque le signal *restart* est activé.

3) L'architecture décrite dans le schéma RTL précédent est retranscrite en code VHDL, définissant le module *Counter_cycle* à l'intérieur du nouveau fichier VHDL *counter_cycle.vhd*.

En plus des entrées et sorties visibles sur le schéma, on définit pour notre module deux paramètres *generic()* :

- Un paramètre *cible* (stocké comme positive, sur 31 bits), qui est utilisé pour la définition du paramètre éponyme dans l'instance du *Counter_unit* interne, et qui va donc définir la durée en nombre de périodes d'horloge des cycles qu'on va compter.
- Un paramètre *size* (stocké comme un positive entre 1 et 32, donc sur 5 bits) qui définit la taille en nombre de bits du signal compteur *n_cycle*.

4) On établit un testbench du module *Counter_cycle* à l'intérieur du fichier de simulation *tb_counter_cycle.vhd*.

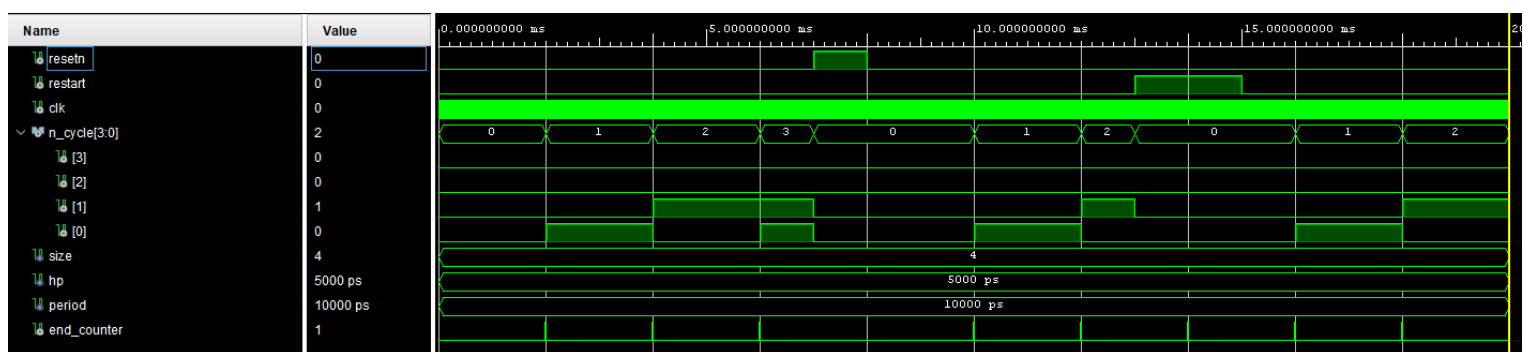
Pour rendre notre simulation rapide, on va repartir des paramètres utilisés pour le TP2 :

- Une horloge de fréquence 100 MHz, soit une période de 10 ns, avec un signal carré (demi-période 5 ns)
- Une *cible* du compteur de 200000, ce qui correspond à des cycles de 2 ms

Pendant notre simulation, on va observer principalement :

- Le signal de sortie *n_cycle*, notre compteur de cible.
- Le signal interne *end_counter*, qui forme un pic à chaque fois que le *Counter_unit* termine un cycle.
- Le signal d'entrée *resetn*, dont la mise en état haut doit réinitialiser à 0 la valeur de *n_cycle*, et dont la remise à l'état bas fait redémarrer le cycle de *Counter_unit* et donc également notre compteur de cycles.
- Le signal d'entrée *restart*, qui tant qu'il est haut réinitialise et bloque à 0 notre compteur de cycles sans arrêter le *Counter_unit*.

Voici le chronogramme observé pour 20 ms de simulation, avec une activation du signal *resetn* entre $t=7$ ms et $t=8$ ms, et une activation de *restart* entre $t=13$ ms et $t=15$ ms :



Pendant les 7 premières millisecondes de fonctionnement normal, on observe bien un pic de *end_counter* toutes les 2 millisecondes, suivi à chaque fois d'une incrémentation de *n_cycle*.

À $t=7$ ms, le signal *resetn* est activé. On observe comme attendu qu'à ce moment, le signal *n_cycle* se réinitialise à 0.

À $t=8$ ms, le signal *resetn* est désactivé. À partir de là, le fonctionnement normal recommence, *end_counter* fait un pic toutes les 2 millisecondes, et *n_cycle* s'incrémente bien après chaque pic.

À $t=13$ ms, le signal *restart* est activé. On observe comme attendu qu'au coup d'horloge suivant, le signal *n_cycle* se réinitialise à 0. On remarque aussi que l'instance *Counter_unit* continue bien de fonctionner alors que *restart* est activé, avec un nouveau pic haut de *end_counter* à $t=14$ ms qui ne déclenche pas d'incréméntation du compteur de cycles.

À $t=15$ ms, le signal *restart* est désactivé. À partir de là, le compteur de cycles est réactivé, et recommencera à s'incrémenter avec le pic haut de *end_counter* à $t=16$ ms.

On a également implémenté des tests automatiques dans notre testbench, pour vérifier que le comportement du compteur de cycles suit bien ce que nous attendons. À noter, notre testbench a été compilé avec la version VHDL 2008, afin de pouvoir effectuer un test sur le signal interne *end_counter*.

Tous ces tests ont été vérifiés :

```
Note: test n_cycle = X'1' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /
Note: test n_cycle = X'2' at 4.000010 ms
Time: 4000010 ns Iteration: 0 Process: /
Note: test n_cycle = X'3' at 6.000010 ms
Time: 6000010 ns Iteration: 0 Process: /
Note: resetn set to '1' at 7 ms
Time: 7 ms Iteration: 0 Process: /tb_cou
Note: test n_cycle = X'0' at 7.000010 ms
Time: 7000010 ns Iteration: 0 Process: /
Note: resetn set to '0' at 8 ms
Time: 8 ms Iteration: 0 Process: /tb_cou
Note: test n_cycle = X'1' at 10.000010 ms
Time: 10000010 ns Iteration: 0 Process:
Note: test n_cycle = X'2' at 12.000010 ms
Time: 12000010 ns Iteration: 0 Process:
Note: restart set to '1' at 13 ms
Time: 13 ms Iteration: 0 Process: /tb_co
Note: test n_cycle = X'0' at 13.000010 ms
Time: 13000010 ns Iteration: 0 Process:
Note: test end_counter = '1' at 14 ms
Time: 14 ms Iteration: 0 Process: /tb_co
Note: test n_cycle = X'0' at 14.000010 ms
Time: 14000010 ns Iteration: 0 Process:
Note: test n_cycle = X'1' at 16.000010 ms
Time: 16000020 ns Iteration: 0 Process:
```

5) Notre machine à états (FSM) comportera quatre états :

- L'état initial *init*, également retrouvé en cas de reset, et dans lequel les 3 LEDs sont activées.
- L'état *red*, dans lequel seule la LED rouge est activée.
- L'état *green*, dans lequel seule la LED verte est activée.
- L'état *blue*, dans lequel seule la LED bleue est activée.

3 signaux viennent piloter notre FSM :

- Un signal d'horloge pour synchroniser nos changements d'états (dans les registres qui vont avec)
- Un signal *resetn* qui vient réinitialiser la FSM dans l'état *init*.

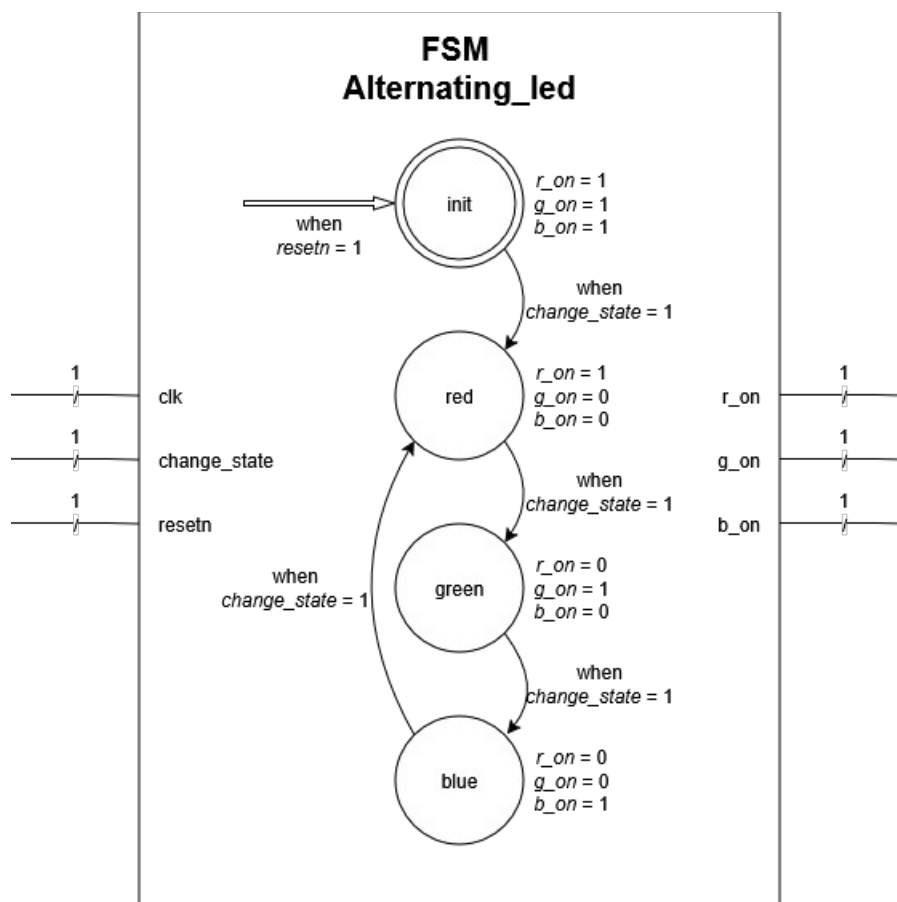
- Un signal *change_state* qui vient déclencher la bascule entre les états (de *init* vers *red*, de *red* vers *green*, de *green* vers *blue*, et de *blue* vers *red*).

Ce signal est constitué pour s'activer tous les 3 cycles de clignotement de LED (en pratique, lorsque *n_cycle* atteint la valeur 6, pour trois cycles de LED « on » et trois cycles de LED « off »).

Notre FSM va produire trois signaux, qui vont chacun permettre ou non à une couleur de LED de s'allumer :

- Le signal *r_on* autorise la LED rouge à clignoter.
- Le signal *g_on* autorise la LED verte à clignoter.
- Le signal *b_on* autorise la LED bleue à clignoter.

Voici le schéma RTL correspondant à cette FSM :



6) La FSM décrite à la question précédente, même couplée à une instance de *Counter_cycle* qui avec une logique conditionnel fournit le signal *change_state*, ne suffit pas directement à fournir les signaux de sorties pilotant les LEDs.

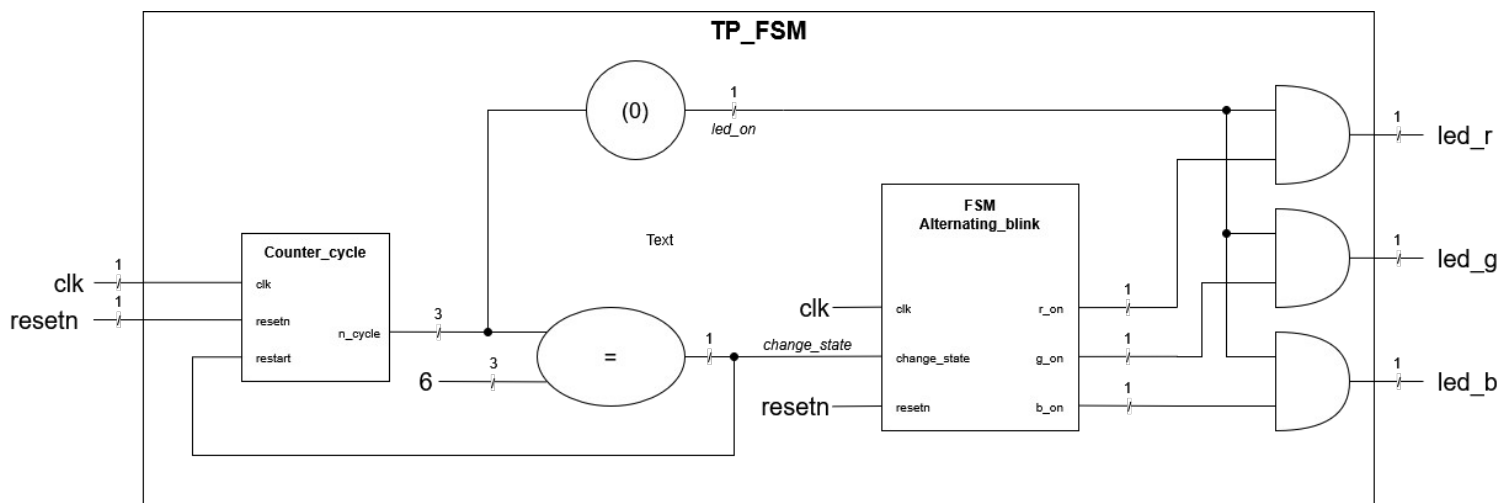
En effet, si la FSM autorise où non chaque LED à clignoter, elle ne fournit pas directement un signal qui permet de piloter le clignotement (qui alterne entre un état haut et un état bas à chaque cycle).

Un tel signal est cependant facilement trouvable en sortie du module *Counter_cycle* : le module fournit la sortie *n_cycle* sur 3 bits (sa valeur ne doit pas dépasser 6), et son dernier bit suit la fréquence voulue pour le clignotement (le bit est haut lors d'un numéro de cycle impair, et bas lors d'un numéro de cycle pair).

Il suffit pour chaque couleur de LED de coupler avec un AND ce signal et celui en sortie de la FSM pour piloter la LED correspondante.

De plus, il faut rajouter une rétroaction supplémentaire pour que le module *Counter_cycle* réinitialise son compte au moment où un changement d'état de la FSM est déclenché. Pour cela, il suffit de relier l'entrée *restart* du module au signal *change_state*.

L'architecture globale a alors ce schéma RTL :



Cette architecture a pour entrées :

- Le signal d'horloge *clk*, sur 1 bit
- Le signal de reset global *resetn*, sur 1 bit (c'est sur cette entrée que sera redirigée le signal du bouton restart, car le comportement de restart attendu doit relancer l'ensemble des compteurs ainsi que la FSM, ce qui est donc équivalent à un reset global)

Elle a pour sorties :

- Le signal *led_r*, sur 1 bit, qui pilote la LED rouge
- Le signal *led_g*, sur 1 bit, qui pilote la LED verte
- Le signal *led_b*, sur 1 bit, qui pilote la LED bleue

Enfin, elle possède plusieurs signaux internes :

- Le signal *n_cycle*, sur 3 bits, obtenu à la sortie éponyme du module *Counter_cycle*
- Le signal *led_on*, sur 1 bit, qui permet de simplifier le pilotage du clignotement des LED, obtenu en récupérant le dernier bit de *n_cycle*

- Le signal *change_state*, sur 1 bit, obtenu par comparaison du signal *n_cycle* avec la valeur 6
- Le signal *r_on*, sur 1 bit, obtenu à la sortie éponyme de la FSM
- Le signal *g_on*, sur 1 bit, obtenu à la sortie éponyme de la FSM
- Le signal *b_on*, sur 1 bit, obtenu à la sortie éponyme de la FSM

7) On implémente en VHDL notre FSM dans un nouveau fichier *tp_fsm.vhd*.

L'architecture décrite comporte :

- Une instantiation de *Counter_cycle* (compteur sur 3 bits, pour pouvoir compter jusqu'à 6)
- La description de la FSM
 - La gestion de l'état de la FSM, avec le reset externe et le déclenchement du changement d'état (avec envoi d'un signal de reset vers le *Counter_cycle*)
 - La configuration des signaux qui autorisent le clignotement de chaque couleur de LED, en fonction de l'état actuel de la FSM
- La logique combinatoire, qui gère :
 - la condition de changement d'état de la FSM en fonction du compteur de cycles en sortie de *Counter_cycle*, qui est également relié à l'entrée *restart* de l'instance *Counter_cycle*
 - le signal de clignotement (dernier bit du compteur de cycles)
 - les signaux de sorties pour le pilotage des LED (combinaisons du signal de clignotement et des signaux d'autorisation par couleur)

8) On écrit un testbench de notre architecture complète dans le fichier *tb_tp_fsm.vhd*.

On va principalement observer :

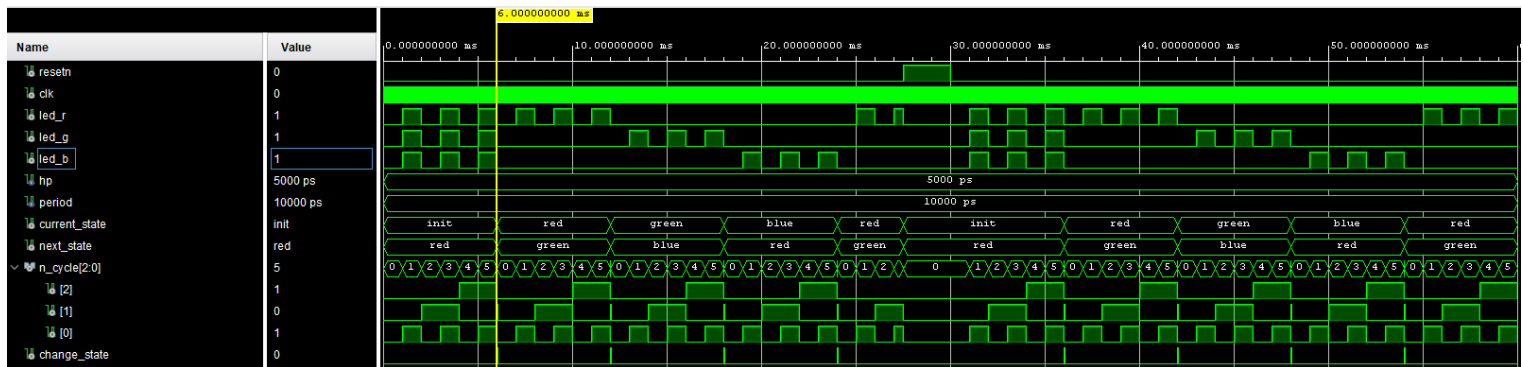
- Nos 3 sorties qui pilotent les LEDs (*led_r*, *led_g*, *led_b*) pour vérifier qu'elles clignotent bien en suivant le motif de la FSM (3 fois blanc → 3 fois rouge → 3 fois vert → 3 fois bleu → 3 fois rouge → 3 fois vert → etc).
- L'entrée *resetrn* qui, lorsqu'elle est activée, doit réinitialiser nos compteurs et la FSM (recommencer à blanc, puis rouge, etc)

On peut aussi observer quelques signaux internes pour vérifier le bon fonctionnement :

- Le signal *current_state*, qui indique l'état actuel de la FSM
- Le signal *next_state*, qui indique le prochain état de la FSM
- Le signal du compteur de cycles *n_cycle*, qui compte de 0 à 6 (cette dernière valeur devant être conservée pendant un unique coup d'horloge)
- Le signal *change_state* qui indique un changement d'état de la FSM, lorsque *n_cycle* vaut 6

À $t = 30$ ms, on relâche le signal *resetrn*, ce qui devrait redémarrer un nouveau fonctionnement normal de notre FSM, en commençant à nouveau par l'état initial blanc.

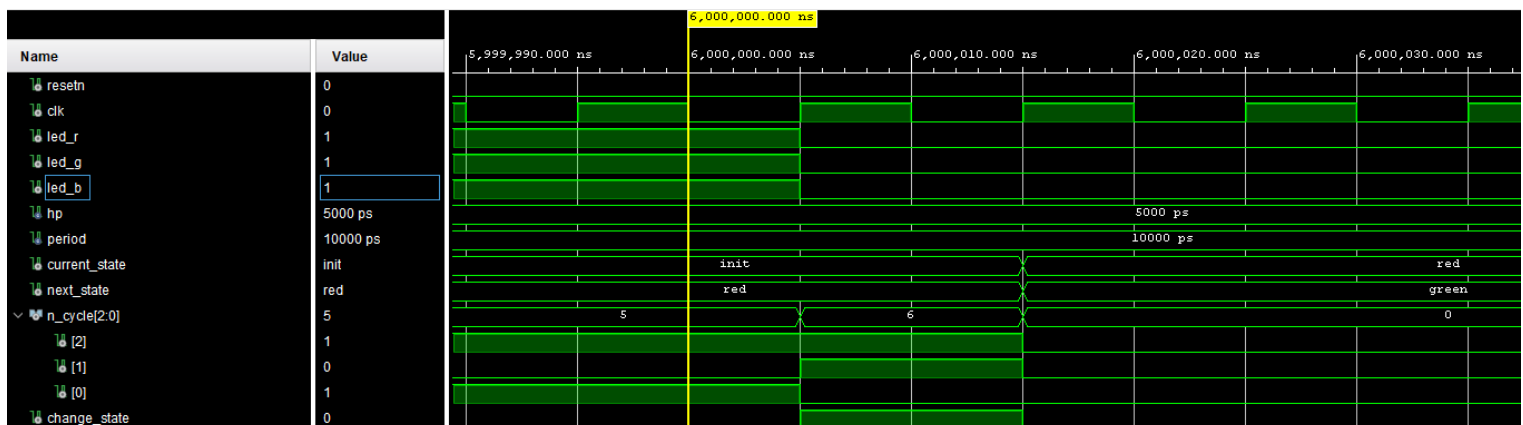
Voici le chronogramme obtenu lorsqu'on simule le testbench décrit précédemment pendant 60 ms :



On observe bien comme prévu le clignotement des LEDs dont les couleurs suivent l'état de la FSM, au rythme du compteur de cycles.

Le comportement du système lorsqu'on active le signal *resetn* correspond aussi à ce qui était attendu : tous les signaux internes et de sortie maintenus à '0' tant que *resetn* est activé, et le retour du fonctionnement normal à partir du moment où *resetn* est désactivé.

Plus précisément, on peut regarder ce qu'il se passe lors d'un changement d'état de la FSM (ici autour de 6 ms) :



Comme prévu, au premier coup d'horloge après $t = 6$ ms, la valeur du compteur de cycles passe de 5 à 6, ce qui déclenche immédiatement par logique combinatoire le signal *change_state*, et éteint les LEDs qui clignotaient (valeur du compteur de cycles paire).

Au coup d'horloge suivant, la FSM va donc changer d'état et passer de *init* à *red*. En même temps, on déclenche le restart de notre compteur de cycles : la valeur *n_cycle* repasse à 0 et par logique combinatoire désactive le signal *change_state*. La valeur de *n_cycle* reste paire, les LEDs restent donc éteintes.

Enfin, au coup d'horloge suivant, puisque le signal *restart* du compteur de cycles est désactivé (*change_state* est repassé à '0'), le compteur de cycles reprend un fonctionnement normal.

On a également implémenté des tests automatiques dans notre testbench, pour vérifier que le comportement des sorties LED suit bien ce que nous attendons.

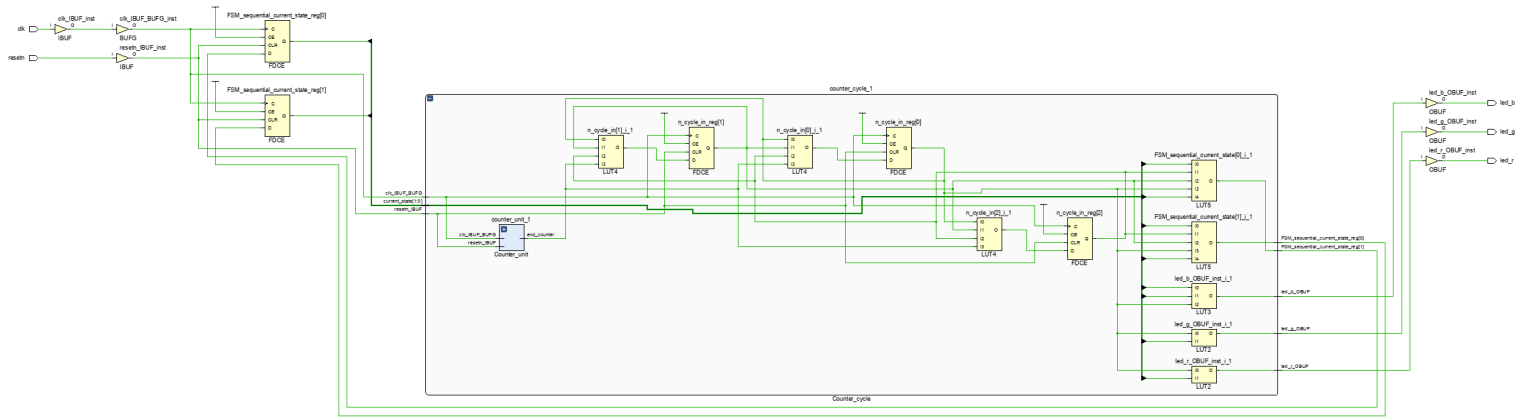
Tous ces tests ont été vérifiés :

```
Note: test led_r = '1' at 1.000010 ms
Time: 1000010 ns Iteration: 0 Process: /t
Note: test led_g = '1' at 1.000010 ms
Time: 1000010 ns Iteration: 0 Process: /t
Note: test led_b = '1' at 1.000010 ms
Time: 1000010 ns Iteration: 0 Process: /t
Note: test led_r = '0' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /t
Note: test led_g = '0' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /t
Note: test led_b = '0' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /t
Note: test led_r = '1' at 7.000010 ms
Time: 7000010 ns Iteration: 0 Process: /t
Note: test led_g = '0' at 7.000010 ms
Time: 7000010 ns Iteration: 0 Process: /t
Note: test led_b = '0' at 7.000010 ms
Time: 7000010 ns Iteration: 0 Process: /t
Note: test led_r = '0' at 8.000010 ms
Time: 8000010 ns Iteration: 0 Process: /t
Note: test led_g = '0' at 8.000010 ms
Time: 8000010 ns Iteration: 0 Process: /t
Note: test led_b = '0' at 8.000010 ms
Time: 8000010 ns Iteration: 0 Process: /t
Note: test led_r = '0' at 13.000010 ms
Time: 13000010 ns Iteration: 0 Process: .
Note: test led_g = '1' at 13.000010 ms
Time: 13000010 ns Iteration: 0 Process: .
Note: test led_b = '0' at 13.000010 ms
Time: 13000010 ns Iteration: 0 Process: .
Note: test led_r = '0' at 19.000010 ms
Time: 19000010 ns Iteration: 0 Process: .
Note: test led_g = '0' at 19.000010 ms
Time: 19000010 ns Iteration: 0 Process: .
Note: test led_b = '1' at 19.000010 ms
Time: 19000010 ns Iteration: 0 Process: .

Note: test led_r = '1' at 25.000010 ms
Time: 25000010 ns Iteration: 0 Process: .
Note: test led_g = '0' at 25.000010 ms
Time: 25000010 ns Iteration: 0 Process: .
Note: test led_b = '0' at 25.000010 ms
Time: 25000010 ns Iteration: 0 Process: .
Note: test led_r = '1' at 27.5 ms
Time: 27500 us Iteration: 0 Process: /tb
Note: At t=27.5 ms, resetn is activated
Time: 27500 us Iteration: 0 Process: /tb
Note: test led_r = '0' at 27.500010 ms
Time: 27500010 ns Iteration: 0 Process: .
Note: At t=30 ms, resetn is deactivated
Time: 30 ms Iteration: 0 Process: /tb_tp
Note: test led_r = '1' at 31.000010 ms
Time: 31000010 ns Iteration: 0 Process: .
Note: test led_g = '1' at 31.000010 ms
Time: 31000010 ns Iteration: 0 Process: .
Note: test led_b = '1' at 31.000010 ms
Time: 31000010 ns Iteration: 0 Process: .
Note: test led_r = '0' at 32.000010 ms
Time: 32000010 ns Iteration: 0 Process: .
Note: test led_g = '0' at 32.000010 ms
Time: 32000010 ns Iteration: 0 Process: .
Note: test led_b = '0' at 32.000010 ms
Time: 32000010 ns Iteration: 0 Process: .
Note: test led_r = '1' at 37.000010 ms
Time: 37000010 ns Iteration: 0 Process: .
Note: test led_g = '0' at 37.000010 ms
Time: 37000010 ns Iteration: 0 Process: .
Note: test led_b = '0' at 37.000010 ms
```

9) On effectue la synthèse sur Vivado.

On obtient la schématique suivante (visible plus clairement dans le fichier *Schématique_TP3.pdf*) :



Sur cette schématique, la partie correspondant à l'instanciation du *Counter_unit* n'est pas développée, car elle correspond à ce qui avait été développé dans le TP2, pour toute la partie compteur et comparateur. La seule différence est le nombre de registre nécessaire à l'enregistrement de la valeur du compteur, qui varie maintenant en fonction de la cible du compteur, et qui nécessite 27 bits dans notre cas où la cible vaut 125 000 000 coups d'horloge, soit 1 seconde.

Sur le reste de la schématique, on observe, de gauche à droite :

- Les entrées, avec les 2 buffers d'entrée IBUF (*resetn_IBUF_inst* et *clk_IBUF_inst*) et le buffer d'horloge BUFG (*clk_IBUF_inst*)
- 2 registres FDCE, qui stockent les 2 bits de l'état de la FSM (*FSM_sequential_current_state_reg[0]* et *FSM_sequential_current_state_reg[1]*)
- Le bloc nommé *counter_cycle_1* qui comporte à la fois
 - Les composants qui constituent l'instance de *Counter_cycle*, c'est à dire :
 - L'instance de *Counter_unit* (non développée ici, comme expliquée précédemment).
 - 3 registres FDCE (*n_cycle_in_reg[0]* à *n_cycle_in_reg[2]*) qui stockent les 3 bits du signal compteur *n_cycle*.
 - 3 tables de vérité de taille 4 LUT4 (*n_cycle_in[0]_i_1* à *n_cycle_in[2]_i_1*) qui permettent le calcul de l'actualisation de chaque bit de *n_cycle*. Chacune de ces LUT4 prends en entrée les 3 bits de *n_cycle* (ancienne valeur) et le signal *end_counter*.

On remarquera que le cas de restart du compteur est bien inclus dans ces LUT, puisque le restart est déclenché par un état particulier de *n_cycle*.
 - Le reste des composants qui constituent la FSM :
 - 2 tables de vérité de tailles 5 LUT5 (*FSM_sequential_current_state[0]_i_1* et *FSM_sequential_current_state[1]_i_1*) qui calculent l'état suivant de la FSM (grâce aux deux entrées de la LUT5 connectées au 2 bits de l'état actuel de la FSM) lorsque *n_cycle* (dont les 3 bits sont aussi en entrées de la LUT5) est égal à 6, et garde l'état actuel sinon.
 - 3 tables de vérité de taille 2 ou 3 (1 LUT3, *led_b_OBUF_inst_i_1*, et 2 LUT2, *led_g_OBUF_inst_i_1* et *led_r_OBUF_inst_i_1*) qui activent les sorties LEDs

lorsque $n_cycle(0)$ (1 des entrées de chacune de ces LUT) est haut et en fonction de l'état de la FSM (en fonction des couleurs, peut se décider avec un seul ou les 2 bits de l'état).

- Les 3 sorties, avec les 3 buffers de sorties OBUF (*led_b_OBUF_inst*, *led_g_OBUF_inst* et *led_r_OBUF_inst*).

Le rapport de synthèse (voir fichier complet *tp_fsm.vds*) indique bien qu'une FSM a été détectée dans notre architecture :

```
INFO: [Synth 8-802] inferred FSM for state register 'current_state_reg' in module 'tp_fsm'
```

State	New Encoding	Previous Encoding
init	00	00
red	01	01
green	10	10
blue	11	11

Le rapport donne les ressources utilisées suivantes :

Report Cell Usage:

	Cell	Count
1	BUFG	1
2	CARRY4	7
3	LUT2	30
4	LUT3	2
5	LUT4	6
6	LUT5	2
7	LUT6	3
8	FDCE	32
9	IBUF	2
10	OBUF	3

En plus des composants explicités dans la description de la schématique, le rapport dénombre aussi les ressources nécessaires à l'instance de *Counter_unit*, c'est à dire 27 registres FDCE pour les 27 bits du compteur (pour un total de 32 avec les 5 explicités précédemment), 7 CARRY4 et 28 LUT2 pour l'incréméntation/reset du compteur (pour un total de 30 LUT2 avec les 2 explicitées précédemment), 1 LUT3 + 3 LUT4 + 3 LUT6 pour la comparaison du compteur avec la cible (pour un total de 2 LUT3, 6 LUT4 avec la LUT3 et les 3 LUT4 explicitées précédemment).

10) Dans notre fichier de contraintes *Cora-Z7-10-Master.xdc*, on ajoute :

- Les deux lignes pour définir notre horloge

```
set_property -dict {PACKAGE_PIN H16 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -period 8.000 -name sys_clk_pin -waveform {0.000 4.000} -add [get_ports clk]
```

- Une ligne pour chacune des 3 sorties LED

```
set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [get_ports { led_b }]; #IO_0_35 Sch=led1_b
set_property -dict { PACKAGE_PIN L14   IOSTANDARD LVCMOS33 } [get_ports { led_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g
set_property -dict { PACKAGE_PIN M15   IOSTANDARD LVCMOS33 } [get_ports { led_r }]; #IO_L23N_T3_35 Sch=led1_r
```

- Une ligne pour l'entrée du bouton de reset

```
set_property -dict { PACKAGE_PIN D19   IOSTANDARD LVCMOS33 } [get_ports { resetn }]; #IO_L4P_T0_35 Sch=btn[1]
```

On notera qu'on a ajouté un deuxième fichier de contraintes (*TP3_Debug.xdc*) qui comporte les éléments nécessaires automatiquement générés pour la gestion debug des sondes ILA.

11) On lance l'implémentation sur Vivado.

On analyse le rapport de timing (voir fichier complet *tp_fsm_timing_summary_routed.rpt*).

Le récapitulatif de timing obtenu est le suivant :

Design Timing Summary											
WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing Endpoints	THS Total Endpoints	WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
1.902	0.000	0	4242	0.021	0.000	0	4226	2.750	0.000	0	2416

On observe notamment que :

- la valeur de *Worst Negative Slack (WNS)* vaut 1,902 ns, ce qui est positif et indique qu'il n'y a pas de violation de set up.
- La valeur de *Worst Hold Slack (WHS)* vaut 0,021 ns, ce qui est positif et indique qu'il n'y a pas de violation de hold.

Le récapitulatif d'horloge est le suivant :

Clock Summary			
Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
dbg_hub/inst/BSCANID.u_xsdbm_id/SWITCH_N_EXT_BSCAN.bscan_inst/SERIES7_BSCAN.bscan_inst/TCK	{0.000 16.500}	33.000	30.303
sys_clk_pin	{0.000 4.000}	8.000	125.000

L'horloge système *sys_clk_pin* est bien configurée selon nos spécifications initiales : période de 8 ns (soit une fréquence de 125 MHz), et signal carré (même demi-période de 4 ns pour les crêtes haute et basse du signal).

Le chemin critique présenté par le rapport est le suivant :

```

-----
From Clock: sys_clk_pin
To Clock: sys_clk_pin

Setup :          0 Failing Endpoints, Worst Slack      1.902ns, Total Violation      0.000ns
Hold  :          0 Failing Endpoints, Worst Slack      0.021ns, Total Violation      0.000ns
PW   :          0 Failing Endpoints, Worst Slack      2.750ns, Total Violation      0.000ns
-----

```

Max Delay Paths

```

-----
Slack (MET) :          1.902ns (required time - arrival time)
Source:          <hidden>
                (rising edge-triggered cell FDRE clocked by sys_clk_pin  (rise@0.000ns fall@4.000ns period=8.000ns))
Destination:     <hidden>
                (rising edge-triggered cell SRL16E clocked by sys_clk_pin  (rise@0.000ns fall@4.000ns period=8.000ns))
Path Group:      sys_clk_pin
Path Type:       Setup (Max at Slow Process Corner)
Requirement:     8.000ns (sys_clk_pin rise@8.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay:  5.965ns (logic 1.571ns (26.337%) route 4.394ns (73.663%))
Logic Levels:    4 (LUT4=1 LUT5=1 LUT6=2)
Clock Path Skew: -0.054ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD):  4.861ns = ( 12.861 - 8.000 )
  Source Clock Delay (SCD):        5.306ns
  Clock Pessimism Removal (CPR):    0.391ns
Clock Uncertainty: 0.035ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
  Total System Jitter (TSJ):       0.071ns
  Total Input Jitter (TIJ):        0.000ns
  Discrete Jitter (DJ):            0.000ns
  Phase Error (PE):               0.000ns
-----

```

On remarque que le slack de ce chemin critique est à environ 24 % d'une période. C'est supérieur à la marge de 20 % qu'on veut avoir pour s'assurer d'un bon fonctionnement de notre système.

12) Sur Vivado, on génère le bitstream correspondant à notre implémentation, et on programme notre carte avec.

La vidéo *Démo_FSM.mp4* présente 40 secondes de fonctionnement de la carte programmée.

La vidéo commence au moment où on relâche le bouton de restart, la carte est donc dans son état initial.

Les 25 premières secondes montrent un fonctionnement normal de la carte :

- on commence dans l'état *init* de la FSM, où la LED clignote (1 s off, 1 s on) trois fois en blanc,
- puis passe dans l'état *red* et clignote trois fois en rouge,
- puis dans l'état *green* et clignote trois fois en vert,
- puis dans l'état *blue* et clignote trois fois en bleu,
- avant de repasser à l'état *red*, où on a le temps de voir l'allumage du premier clignotement rouge.

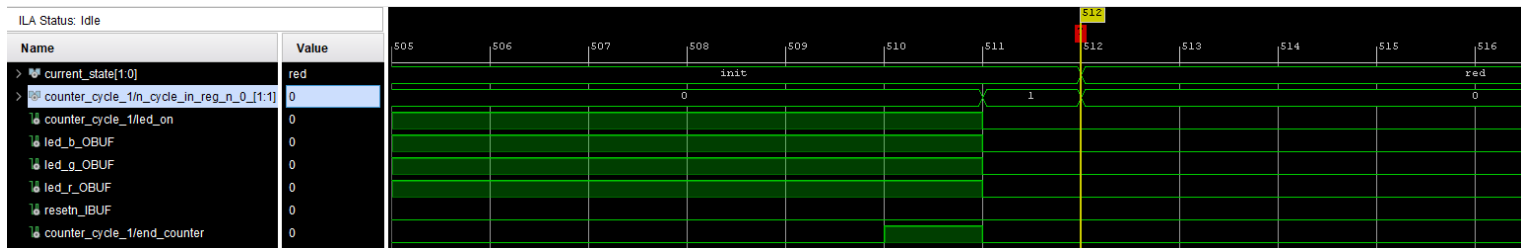
À la 25^{ème} seconde, on presse le bouton restart : la LED, à ce moment allumée en rouge s'éteint immédiatement. La LED reste éteinte tout le temps où le bouton est pressé.

Le bouton est relâché à la 28^{ème} seconde : le fonctionnement normal recommence depuis l'état initial, à la seconde suivante la LED s'allume en blanc. Durant les 12 secondes finales de la vidéo,

on observe à nouveau ce fonctionnement normal, avec 3 clignotements blancs puis 3 clignotements rouges.

En plus de cette évaluation visuel, on effectue plusieurs tests avec la ILA. À noter, un problème de Vivado m'empêche de faire apparaître dans la fenêtre Waveform le bit (2) du signal *n_cycle*.

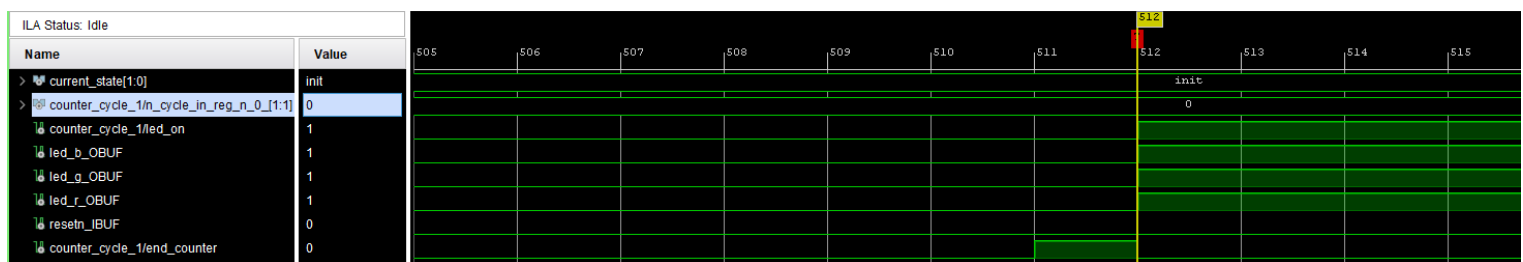
Le premier test est trigger lorsque l'état de la FSM passe à *red*, mis en place pendant un restart :



Comme on l'avait simulé, le changement d'état de la FSM et le reset du compteur de cycles ont lieu un coup d'horloge après que le compteur de cycles atteigne la valeur 6, incrémentation qui a elle même lieu un coup d'horloge après la montée du signal *end_counter*.

On remarquera que l'extinction des LEDs n'a pas lieu au moment exact où l'état de la FSM change, mais au coup d'horloge précédent, quand le compteur de cycles atteint une valeur paire (de 6).

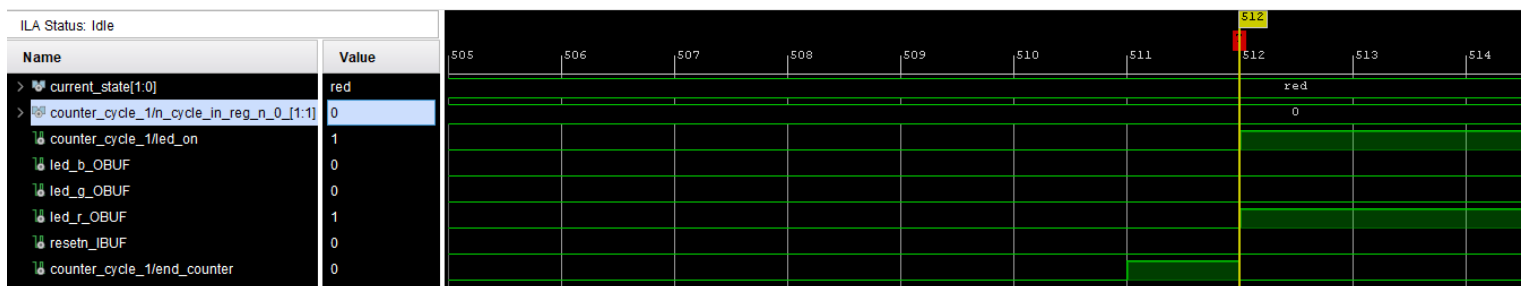
Le second test est trigger par un front montant du signal *led_on*, pendant un cycle d'initialisation :



On observe ici que l'incrémement du compteur de cycles (qui passe de 0 à 1) est déclenchée par le la montée du signal *end_counter*.

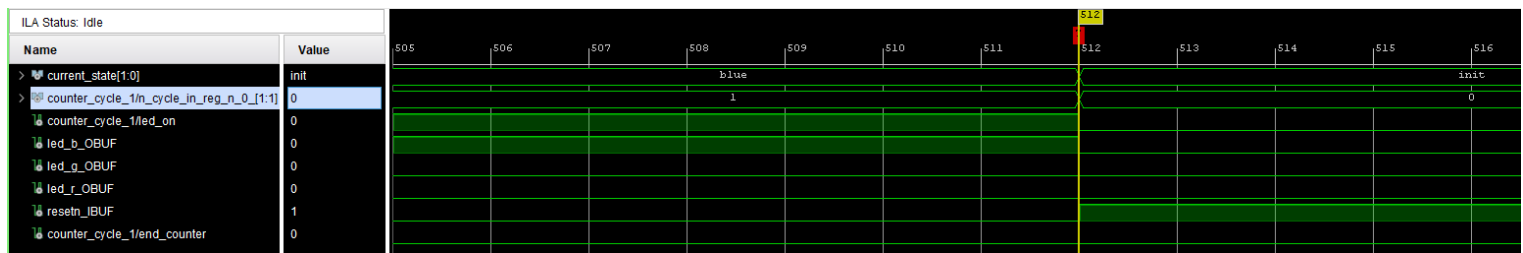
Le passage du compteur de cycles à une valeur impaire active le clignotement des LED. La couleur des LEDs qui s'allument est gouvernée par l'état de la FSM : dans l'état *init*, ce sont toutes les LEDs qui s'allume.

Si on lance un test avec le même trigger alors que la FSM est dans un autre état, le résultat serait différent :



Ici pendant un état *red*, c'est uniquement la LED rouge qui s'allume.

Le test suivant est trigger par un front montant du signal *resetn*, et on va appuyer sur le bouton restart pendant qu'une des LED est allumée.

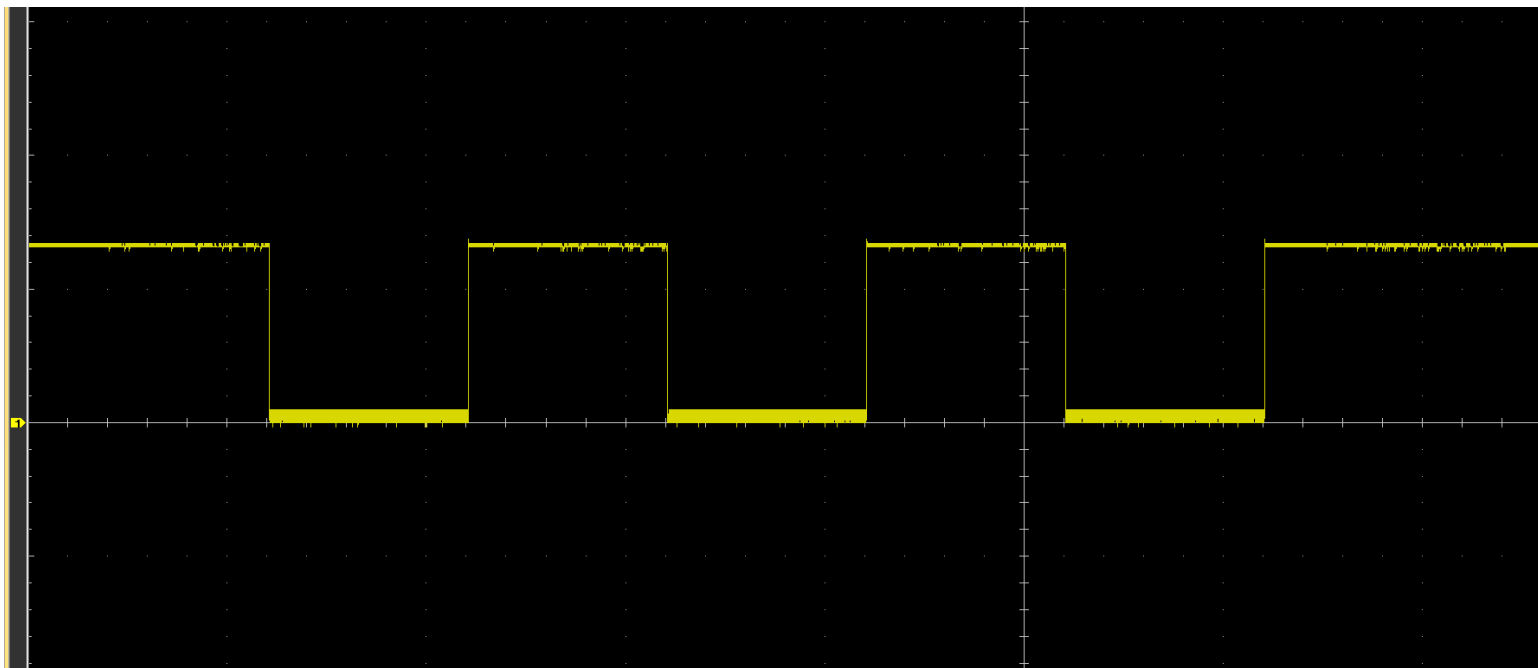


On observe qu'au moment où le bouton est enfoncé, l'ensemble du système se reset : l'état de la FSM repasse à *init*, la LED bleue qui était allumée s'éteint, le compteur de cycles repasse à 0.

Enfin, on vérifie également la durée de nos cycles avec une mesure à l'oscilloscope.

Notre mesure est effectuée au pin d'entrée de la LED bleue. Selon la documentation de notre carte Cora Z7, le signal de sortie de FPGA qui pilote les LED est inversé : lorsque le signal de sortie est haut, il est alors bas en sortie de la LED qui se retrouve sous tension, et donc allumée.

Nous mesurons le signal suivant (division verticale de 2 V, division horizontale de 1 s) :



On observe bien que lorsque la LED bleue est éteinte, le signal à l'entrée de la LED est haut (à 3,3 V, ce qui correspond bien à du LVCMOS33), ce qui correspond à un signal de sortie bas.

Pendant trois périodes de 1 s, notre signal passe à 0 V : cela correspond aux trois clignotements allumés en bleu pour cet état de la FSM, lorsque le signal de sortie est haut.

Sur cet enregistrement de notre signal, on observe que la période allumée de la LED, ainsi que la période éteinte entre deux clignotements valent bien 1 s à chaque fois. Cela confirme notre réglage du couple horloge – paramètre de *cible* de notre *Counter_unit* pour obtenir des cycles de 1 seconde.

Au final, que ce soit en visuel, via l'ILA ou à l'oscilloscope, la carte semble parfaitement fonctionner.